《计组II》

第七章——存储系统(6)

李瑞　　教授

蒋志平　讲师

计算机科学与技术学院

温故 —关于存储系统(1/2)

- 为什么需要Cache，从性能和结构两方面简述
- 什么是局部性原理?
- 什么是存储系统? 典型计算机有哪几个存储系统?
- Cache系统中，地址变换/映射机构的作用? 可能采用的物理器件是?
- 简述全相联映射。其特点?
- 简述直接映射，其特点?
- 简述组相联映射，其特点?

温故 —关于存储系统(2/2)

- 简述Cache替换策略FIFO
- 简述Cache替换策略LRU
- 当某一内存块进入Cache之后又命中一次，FIFO及LRU策略下，这一块被踢出的顺序是否受影响?
- 简述Cache/内存写一致性问题
- 简述写回(Write Back)法
- 简述全写/写直达(Write Through)法
- 写直达方法存在的速度不匹配指什么？ 如何处理?

Cache性能分析与优化

Cache性能分析与优化

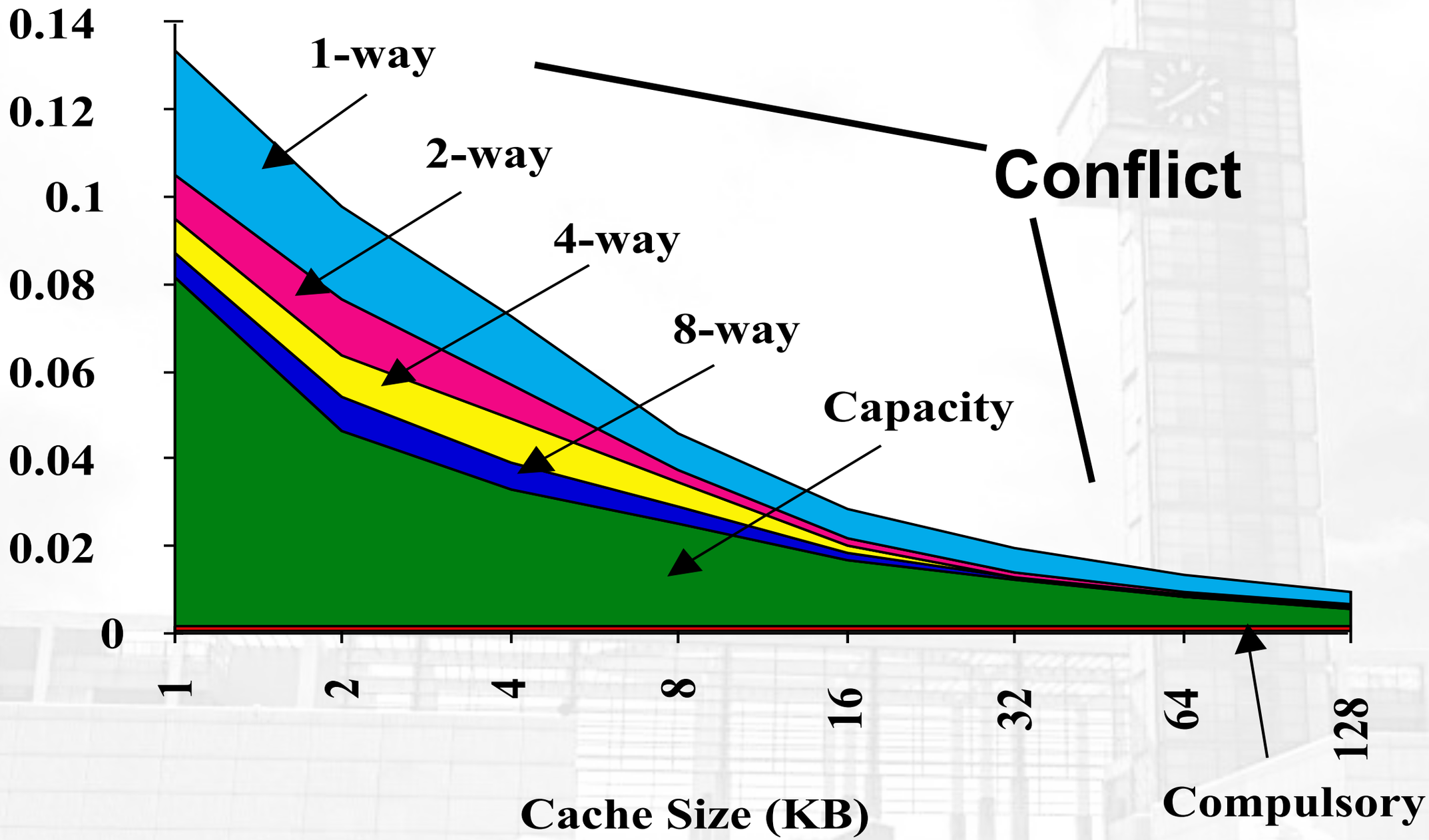
- **Cache命中(Hit)**
  - **CPU欲访问的数据已在缓存中，即可直接访问 Cache**
- **Cache非命中(Miss)**
  - **数据不在Cache，需要将主存数据调入Cache**
  - **CPU被阻塞(Blocking)，等！**
- **命中率(Hit Rate)**
- **非命中率(Miss Rate)**

Cache性能分析:Miss的分类4C

- Miss的4种类型(4C):
  - Compulsory（必然)
    - 冷启动、过程转移、首次引用等不可避免的miss...
  - Capacity（容量）
    - 容量不够造成的
  - Conflict（冲突）
    - Cache块冲突
      - 方案1：增加容量
      - 方案2：提升相联度
  - Coherence（相关）
    - 其它操作带来的结构相关（I/O操作等）

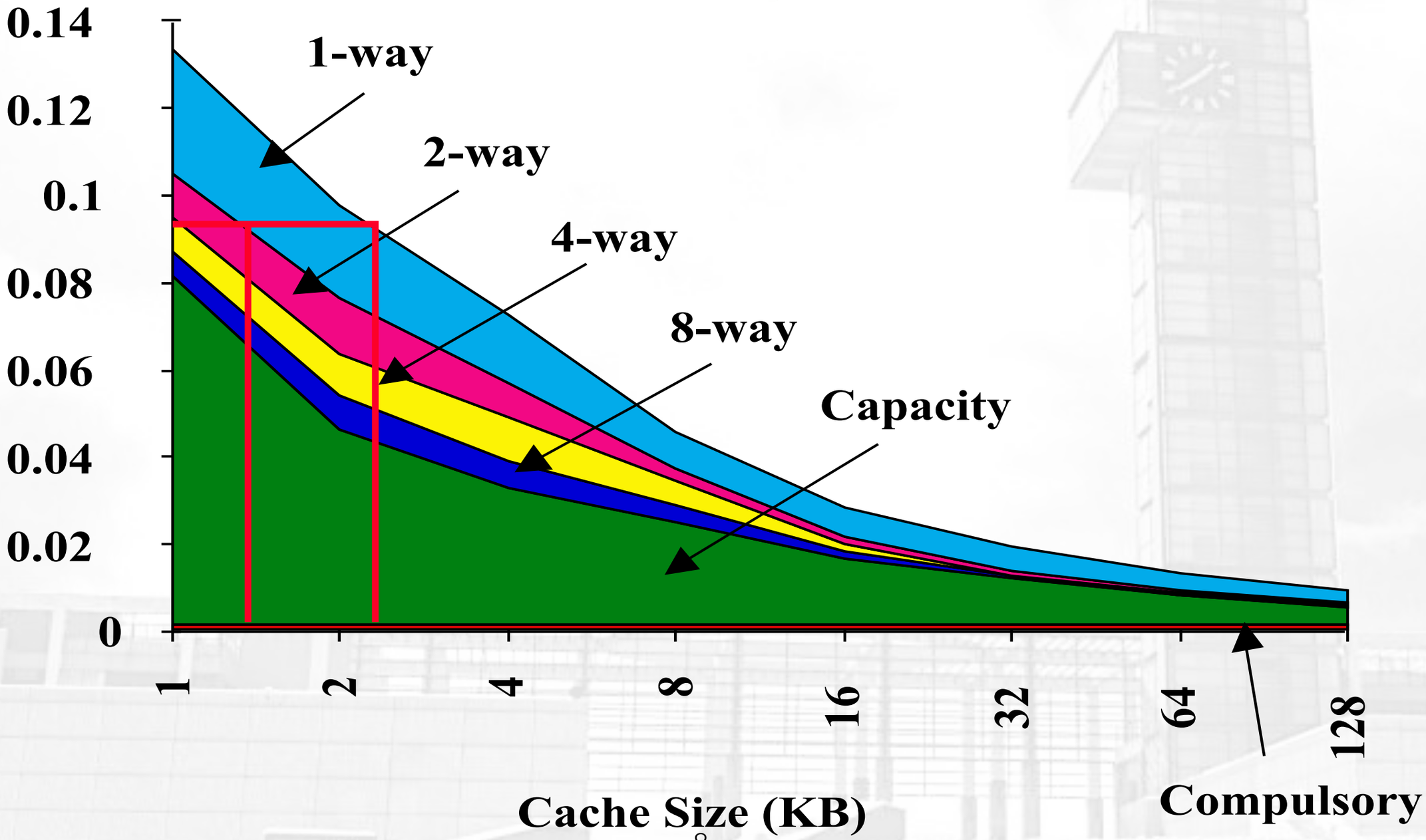
Cache性能分析:Miss的分类4C

- Conflict Miss随容量提升而降低



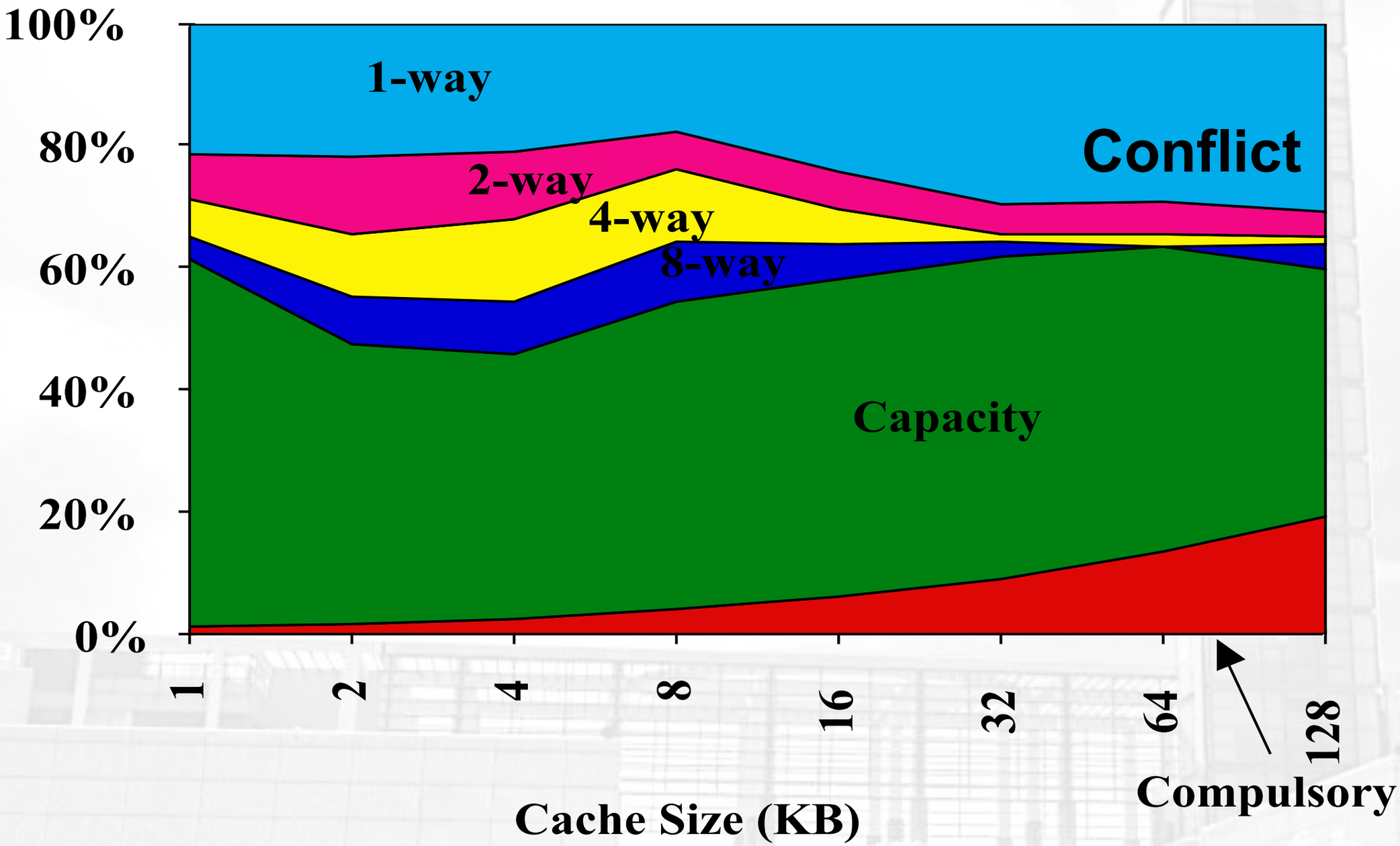
Cache性能分析:Miss的分类4C

- 多路组相联明显降低miss rate
  - 相同的miss rate，1-way组相联(直接映射)需要2倍的Cache size才能达到2-way组相联水平



Cache性能分析:Miss的分类4C

- 3种miss的比例



Cache性能分析与优化

1. 加速比

■ Cache-主存系统的平均访问周期T：

$$T = H \times TC + (1 - H) \times (TB + TC)$$

$$T = TC + (1-H) * TB$$

- Cache的访问周期为TC，主存的访问周期为TM，数据块调入Cache的块传输时间为TB，Cache的命中率为H。

■ Cache-主存系统的加速比SP：SP＝TM/T　<—(Old duration / new)

Cache性能分析与优化

例如，设Cache的速度是主存的5倍，命中率为 95%，则采用Cache后性能提升多少?

系统平均访问时间=*t+0.05*5t=1.25t*

性能提升=*5t/1.25t=4*倍

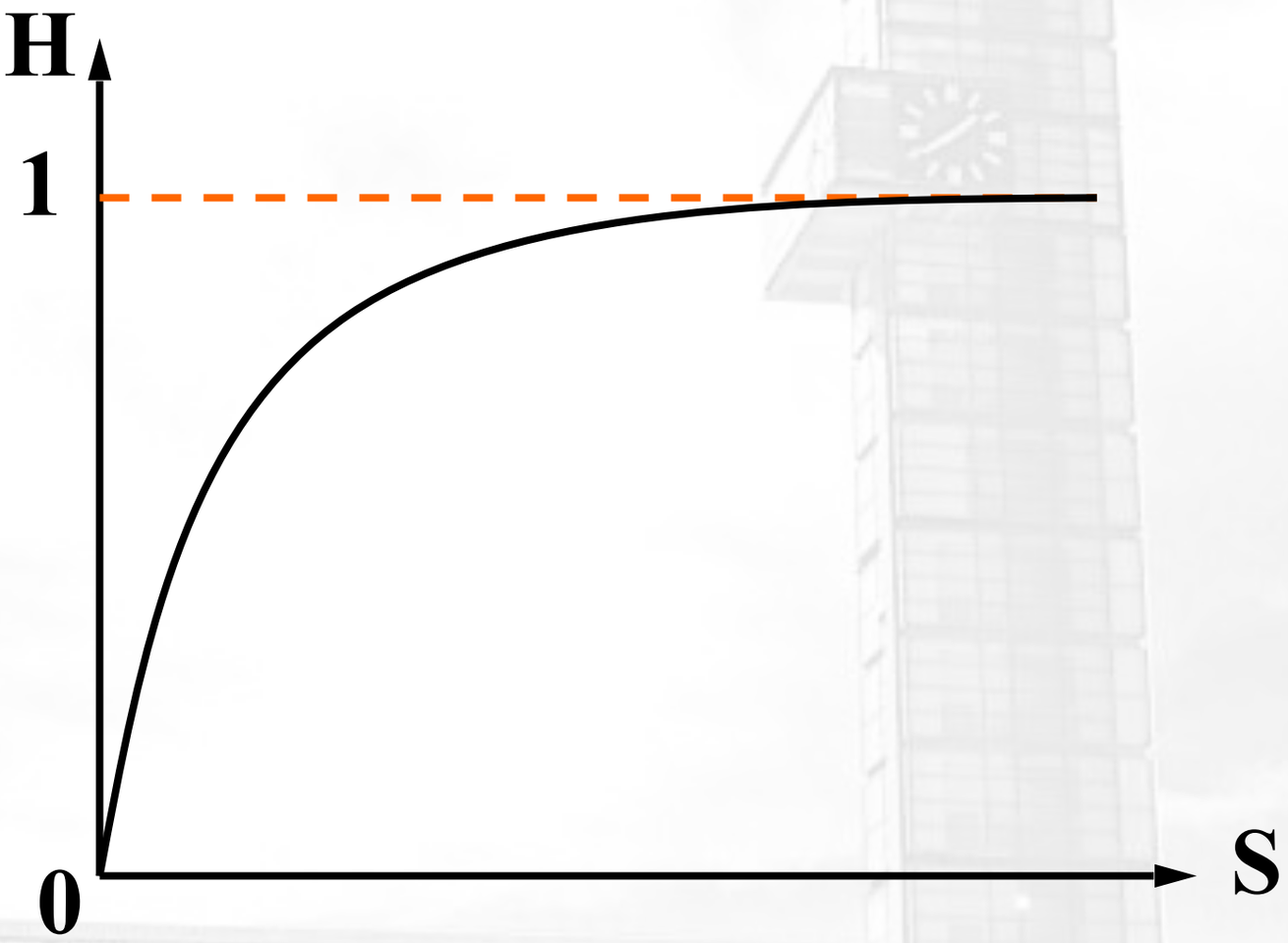
Cache性能分析与优化

- 如何提升Cache性能?
  - 容量
  - 块数/块大小
  - 多路组相联
  - 多级Cache
  - 指令/数据独立Cache
  - Victim Cache
  - 代码层面
  - …

Cache性能分析与优化

优化方案1：提升Cache容量

- 边际效率降低
- 成本问题…
- 制成/热设计…



Cache容量S与命中率H的关系

Cache性能分析与优化

存储系统成本

- **C = (C1×S1 + C2×S2)/(S1 + S2)**

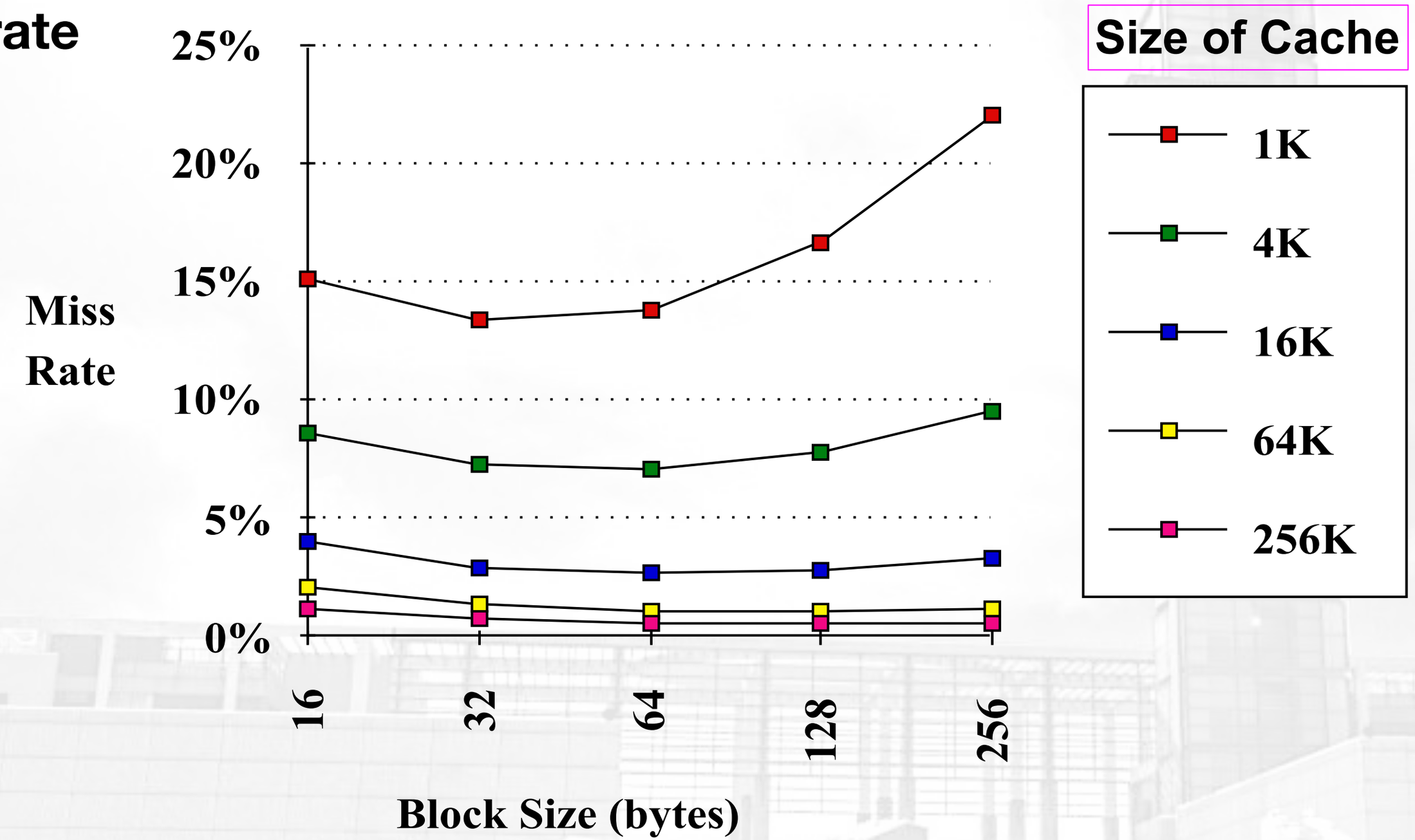
主存价格　主存容量　Cache容量　Cache价格

S1>>S2

- 尽管**Cache**的价格比主存高，但是当其容量很小时，存储器的平均价格**C**接近于主存的价格。
- 由于设置了**Cache**使**CPU**的访存速度接近**Cache**的速度，而使存储器的成本接近于主存的价格。

Cache性能分析与优化

优化方案2：利用时空局部性原理，通过提升块大小，降低miss rate



Cache性能分析与优化

优化方案2：利用时空局部性原理，通过提升块大小，降低miss rate

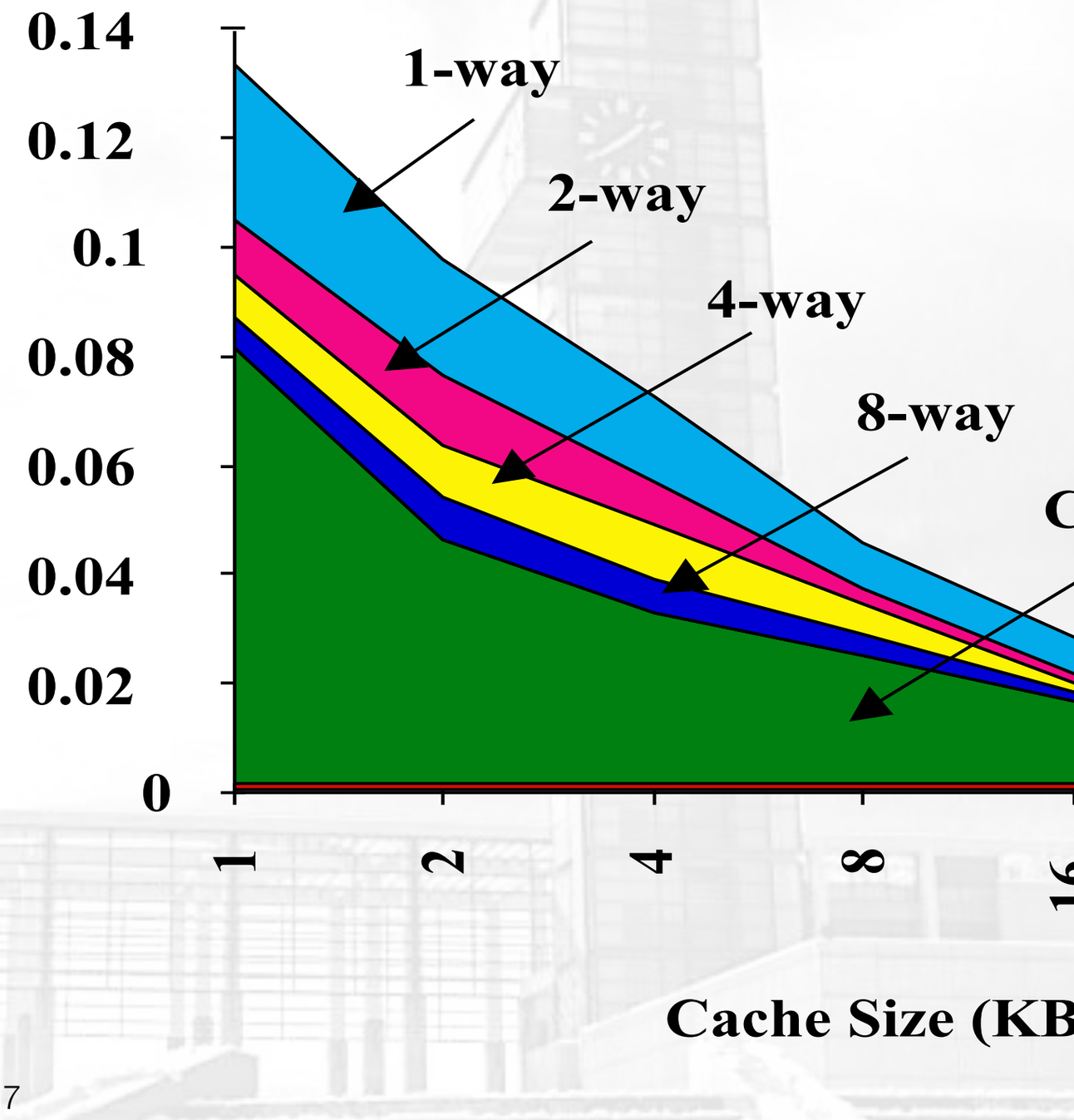
- 降低Compulsory必然miss rate
- 但太大的块有缺点
  - miss的代价大；
  - Conflict冲突导致的miss高

- 总结，差不多大就得了。

Cache性能分析与优化

优化方案3：提升组相联级别

- 级别高一些，还是好的...
  - 2:1规则，2路组相联只需要一半的Cache容量就可以达到1路组相联同等的miss rate/hit rate
  - 但级别提升的边际效率在下降
- 级别也不要太高
  - 相联比较复杂
  - 相联存取复杂—>提高TC时间



Cache性能分析与优化

- 例如—组典型数据…

Cache Size (KB)	Associativity			
	1-way	2-way	4-way	8-way
1	7.65	6.60	6.22	5.44
2	5.90	4.90	4.62	4.09
4	4.60	3.95	3.57	3.19
8	3.30	3.00	2.87	2.59
16	2.45	2.20	2.12	2.04
32	2.00	1.80	1.77	1.79
64	1.70	1.60	1.57	1.59
128	1.50	1.45	1.42	1.44

红色说明8路组相联已经没有性能提升

Cache性能分析与优化

优化方案4：多级Cache

- 两级**Cache**的总未命中率(总失效率)：

  总失效率＝(失效率)$_{第一级}$×(失效率)$_{第二级}$.....×(失效率)$_{第N级}$

- 【例】**10000**次访存，第一级**Cache**失效**400**次，第二级**Cache**失效**4**次。
  - (失效率)$_{第一级}$＝**400/10000**＝**4**%
  - (失效率)$_{第二级}$＝**4/400**＝**1**%
  - 利用两级**Cache**后总的失效率＝**0.04**%。

Cache性能分析与优化

5. 多级Cache的命中率

【例】访问内存需50ns, L1 1ns 10%失效率，L2 5ns 1%失效率，L3 10ns 0.2%失效率，求L1， L1+L2，L1+L2+L3构架下的平均访问时间。

L1 ： $T = 1\ ns + (0.1 \times 50\ ns) = 6\ ns$

L1+2 ： $T = 1\ ns + (0.1 \times [5\ ns + (0.01 \times 50\ ns)]) = 1.55\ ns$

L1+2+3 : $T = 1\ ns + (0.1 \times [5\ ns + (0.01 \times [10\ ns + (0.002 \times 50\ ns)])]) = 1.5001\ ns$

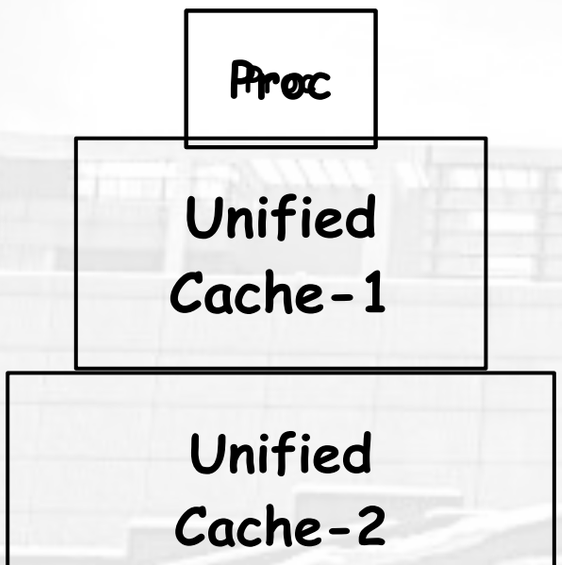
Cache性能分析与优化

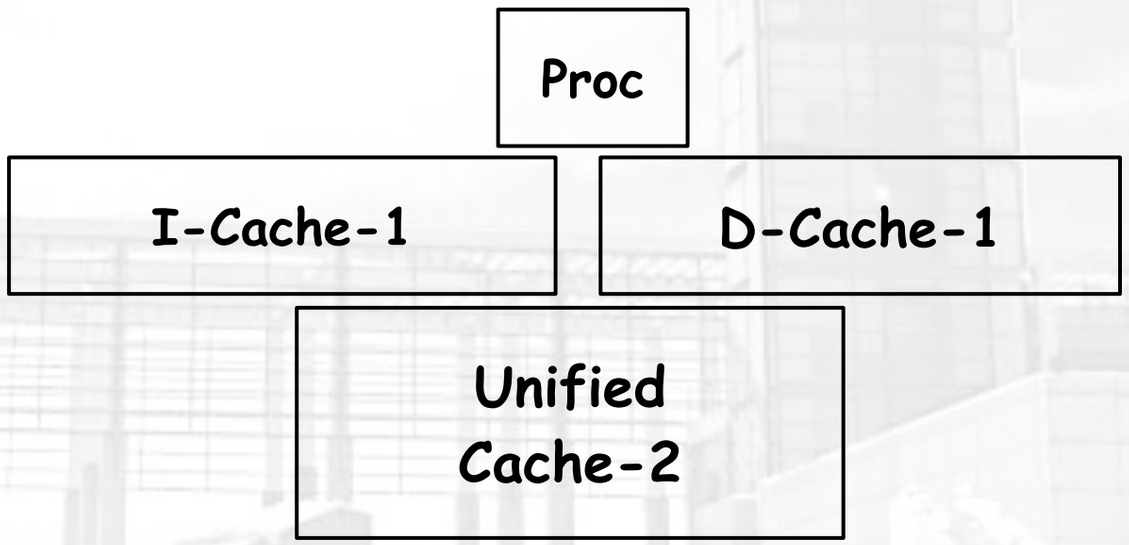
优化方案5：拆分出指令/数据独立Cache

- 例如
  - 16KB 指令/数据独立Cache
    - 指令 miss rate=0.64%, 数据 miss rate=6.47%
  - 32KB 统一Cache: miss rate=2.99%
  - 假设75%指令,25%数据， Hit time = 1, Miss time = 50

T独立=75%x(1+0.64%x50)+25%x(1+6.47%x50) = 2.05

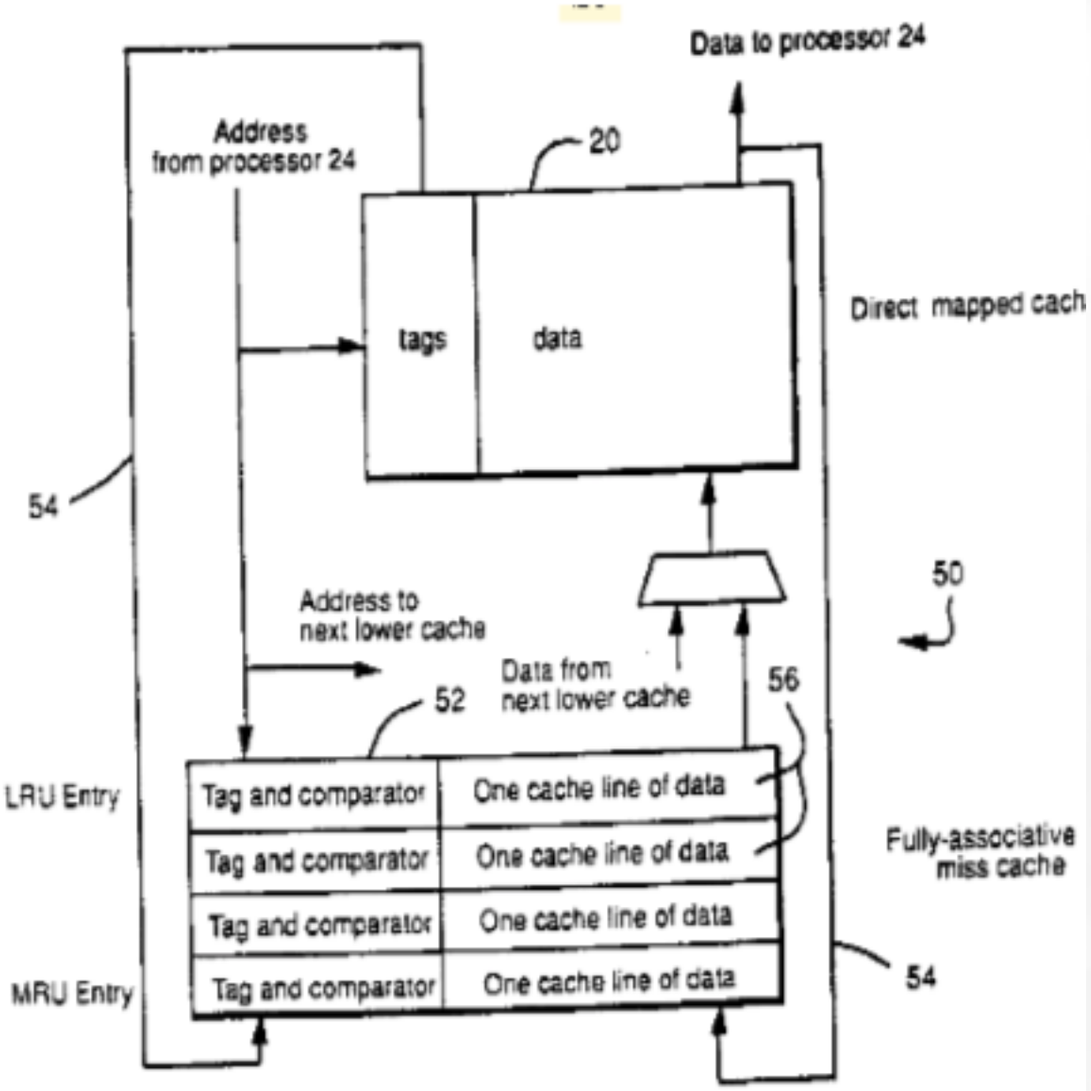
T统一=75%x(1+2.99%x50)+25%x(1+2.99%x50)= 2.50





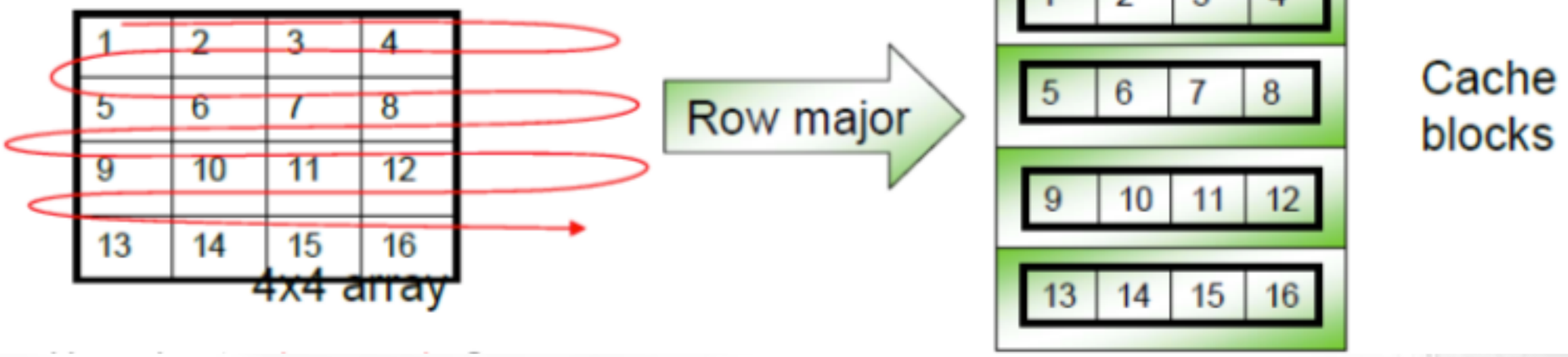
Cache性能分析与优化

- 优化方案6：Victim Cache
- Miss的代价很大，如何优化?
  - Victim Cache，在L1和L2/主存之间
- Victim Cache容量小，采用全相联
- 例如，4个块的Victim Cache可以降低4KB直接映射miss rate 20-90%。



Cache性能分析与优化

- 优化方案7：代码层面: 1) Row Major
- 注意循环：大部分计算和访存都在循环中发生
  - 按照数据对象在存储器中的存储顺序读取，最优
  - 否则，连续miss



- 原程序(Column Major)

```
a[100][5000]=...//初始化
for(j=0; j<5000; j=j+1) {
  for(i=0; i<100; i=i+1) {
    a[i][j] = 2 * a[i][j]; 每次都不命中
  }
}
```

- 优化后(Row Major)

```
a[100][5000]=...//初始化
for(i=0; i<100; i=i+1) {
  for(j=0; j<5000; j=j+1) {
    a[i][j] = 2 * a[i][j]; 可连续命中若干次[cache行大小]
  }
}
```

Cache性能分析与优化

- 优化方案7：代码层面: 2) 循环合并

```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1)
        a[i][j] := 1 / b[i][j] * c[i][j];
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1)
    d[i][j] := a[i][j] + c[i][j];
```

- 原程序中a和c访问两次
- 有可能不在cache中

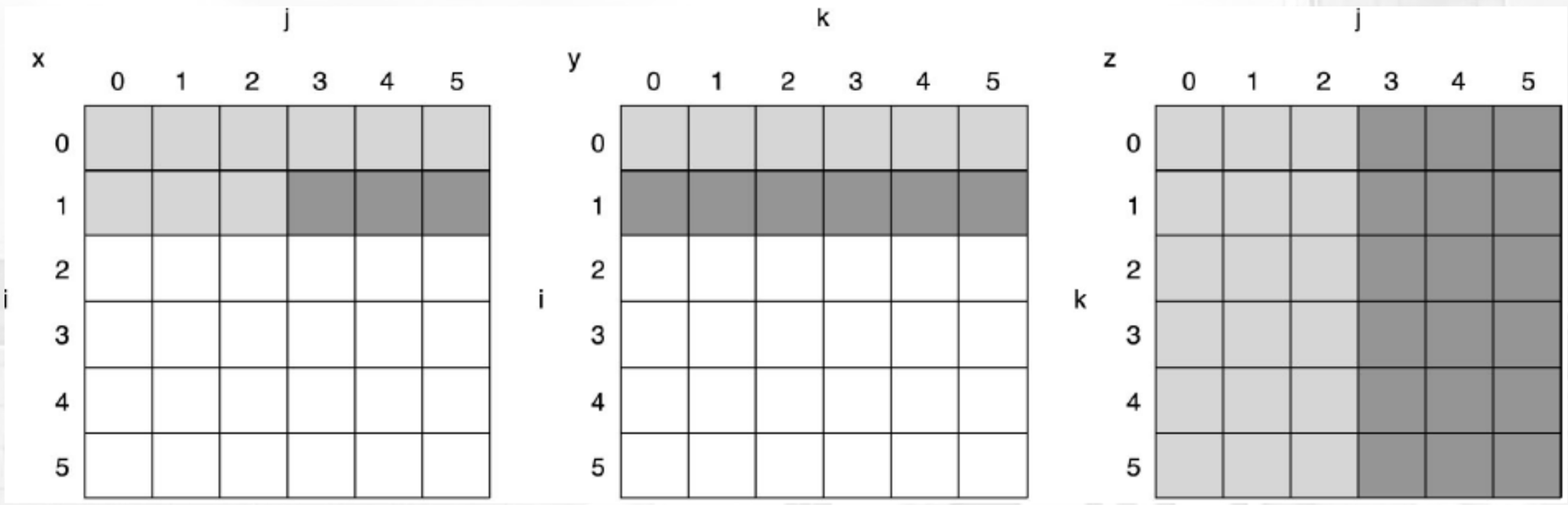
```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1){
        a[i][j] := 1 / b[i][j] * c[i][j];
        d[i][j] := a[i][j] + c[i][j];}
```

优化后a和c第2次访问都在cache里

Cache性能分析与优化

- 优化方案7：代码层面: 3) 分块运算
- 例如，矩阵.*运算
  - 当3个矩阵无法全放在cache时，严重的Capacity Miss

```
for(i := 0; i < N; i := i+1)
    for(j := 0; j < N; j := j+1){
        r := 0;
        for(k := 0; k < N; k := k+1)
            r := r + y[i][k] * z[k][j];
x[i][j] := r;}
```

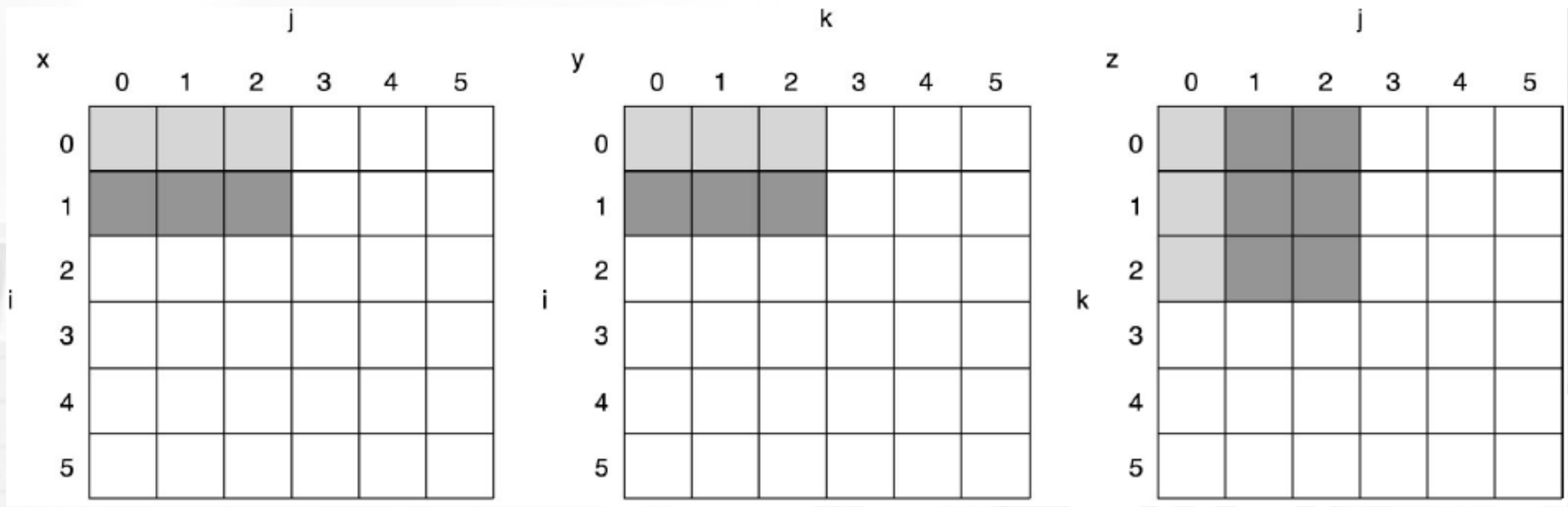


Cache性能分析与优化

- 优化方案7：代码层面: 3) 分块运算
- 例如，矩阵.*运算
  - 当3个矩阵无法全放在cache时，严重的Capacity Miss
  - 把矩阵分成小块执行，充分避免Capacity Miss

After:

```
for(jj := 0; jj < N; jj := jj+B)
for(kk := 0; kk < N; kk := kk+B)
for(i := 0; i < N; i := i+1)
for(j := jj; j < min(jj+B-1,N); j := j+1){
    r := 0;
    for(k := kk; k < min(kk+B-1,N); k := k+1)
        r := r + y[i][k] * z[k][j];
    x[i][j] := x[i][j] + r;}
```



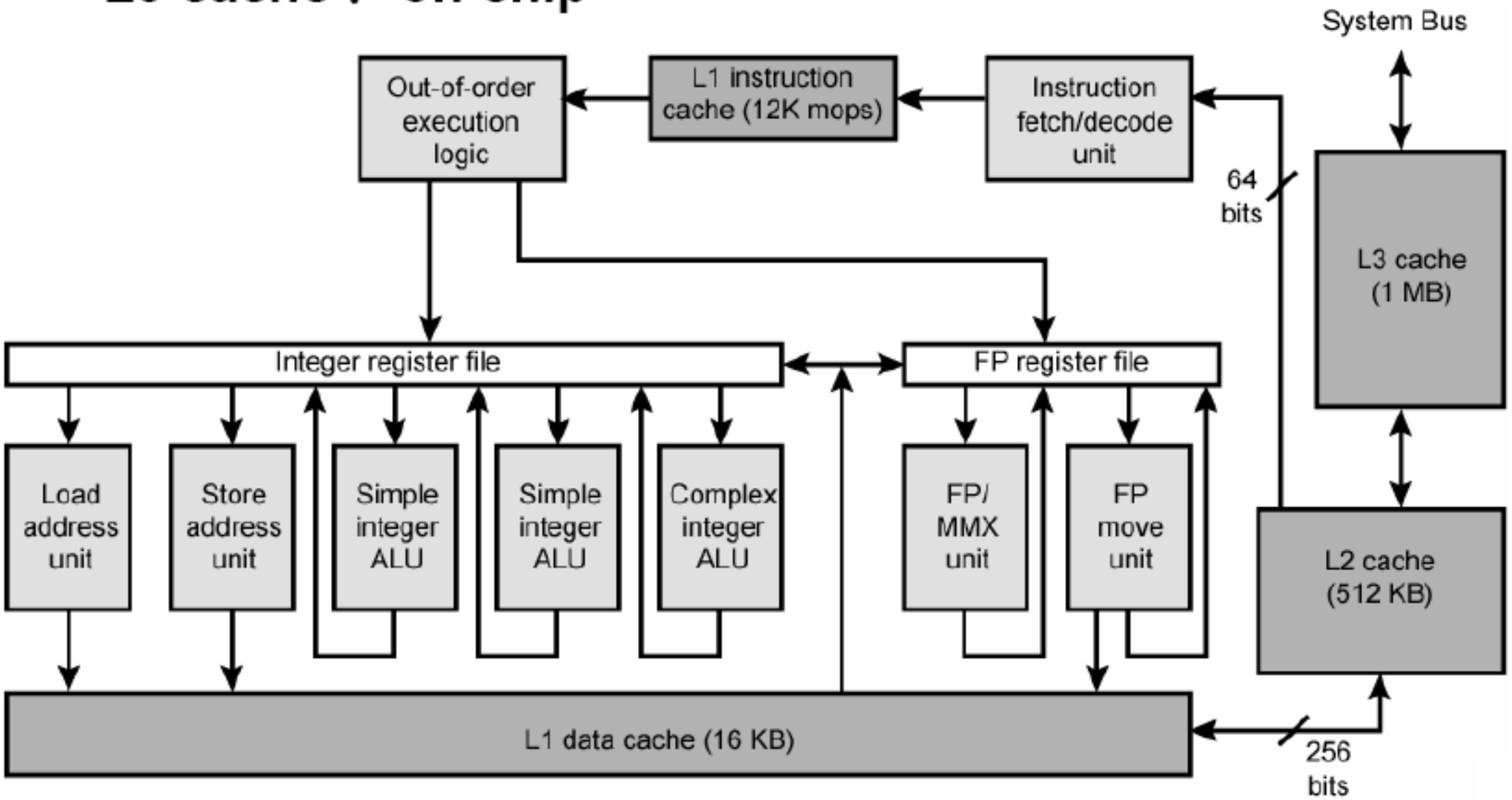


On Spectre Vulnerability

Spectre攻击技术

- 背景知识：
  - CPU早已是乱序执行构架（OoO-E），而且预测执行（Speculative Execution），原理很简单：
    - 多路分支同时执行（同时执行if/switch/for的多分支）
    - 但结果临时存放在CPU中，并不写回内存
    - 直到确定哪条分支真正被执行，才写内存，丢弃其它分支结果
- 看似是完美的性能倍增技术：性能提升，正确性有保证

- **Pentium 4 Cache**
  - L1 caches: 64 byte lines，4 way set associative
  - L2 cache: 128 byte lines，8 way set associative
  - L3 cache : on chip



Spectre攻击技术

- 但预测执行还是会有“痕迹”的
  - 不同分支访问内存，Cache会留痕迹
    - 痕迹：有些并未被正确分支使用到的内存块被加速了（放入cache）！
- 这就可以被用来实施攻击了！

Spectre攻击技术

- 如何攻击？攻击什么？
  - 在正常情况下，有些权限操作是不能被执行的，比如通过通过系统的特殊API访问其它进程的内存...
  - 但在预测执行时，其实这些API被真实地调用过！
    - 结果并没有写回内存
    - 如何获取这些没有写回内存的结果？
      - cache的加速痕迹！

Spectre攻击技术

- 如何攻击？攻击什么？
  - 在正常情况下，有些权限操作是不能被执行的，比如通过通过系统的特殊API访问其它进程的内存...
- 但在预测执行时，其实这些API被真实地调用过！
  - 结果并没有写回内存
  - 如何获取这些没有写回内存的结果？
    - cache的加速痕迹！
      - 访问一处根据API返回结果转变出的内存！

Spectre攻击技术

- 如何获得最后的结果?
  - 扫描内存!
    - 检查每一个块的访问速度!
      - 被访问过的快访问速度快，反之慢。
- 得到一张内存访问速度表
  - 反求出权限API返回的结果!

虚拟存储器

Virtual Memory, VM

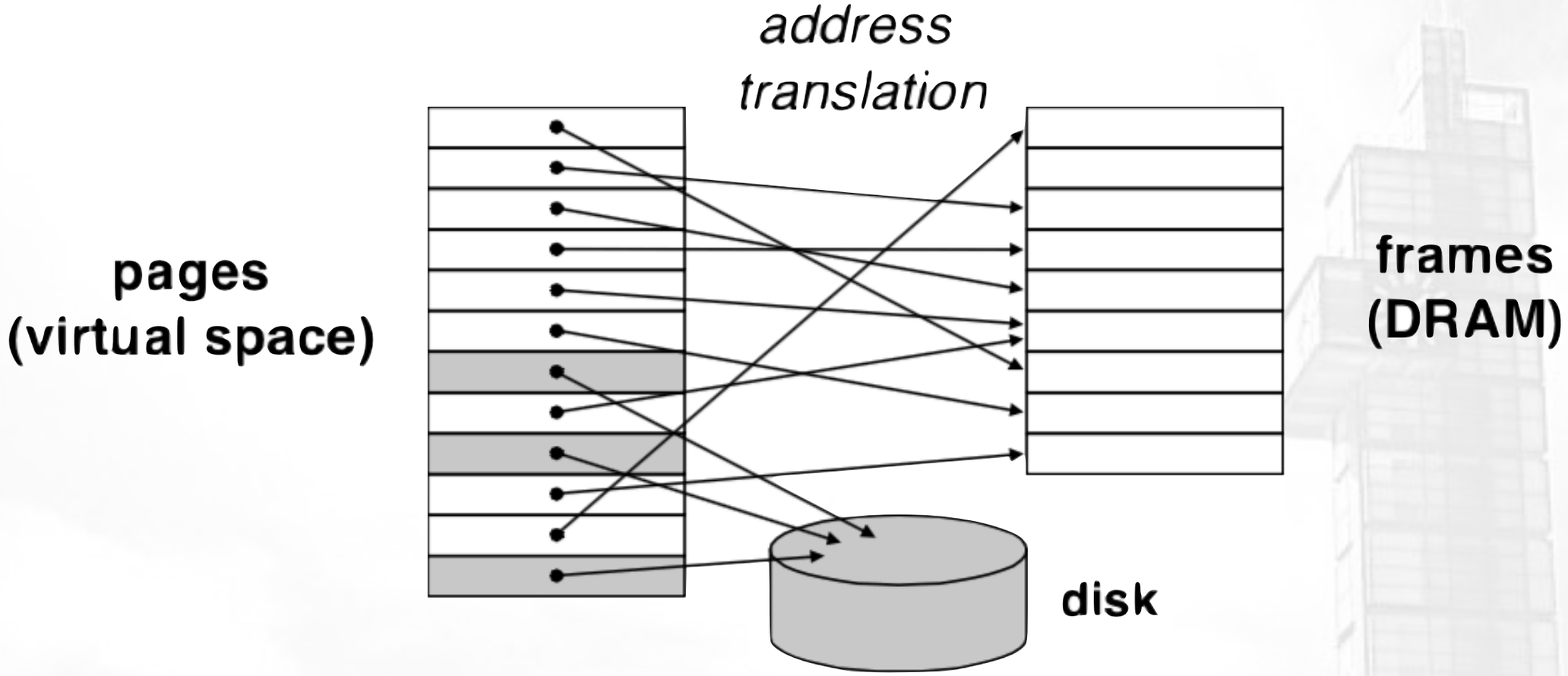
VM：两种地址空间

- 虚拟地址空间
  - 每个进程独占的“巨大 + 连续”的虚拟内存上的地址
- 物理地址空间
  - 实际的物理内存上的地址
  - 被操作系统+硬件完全屏蔽

VM的原理

- 1. 把内存（物理&虚拟）分为同样大小的块或段
- 2. 每个进程在各自的互相隔离的VM中占用一些内存块或段
- 3. 通过“地址变换表”Address Translation Table，将VM中的块或段映射到物理内存
- 虚拟地址<—>物理地址变换由硬件完成

VM的原理



- 某种角度来说，内存成了外存的Cache
  - 实现“进程连续并且更大”的内存
  - 程序在内存中的实际位置无意味
  - 进程隔离、进程共享、内存动态管理等优势

存储系统

存储系统的多层次结构

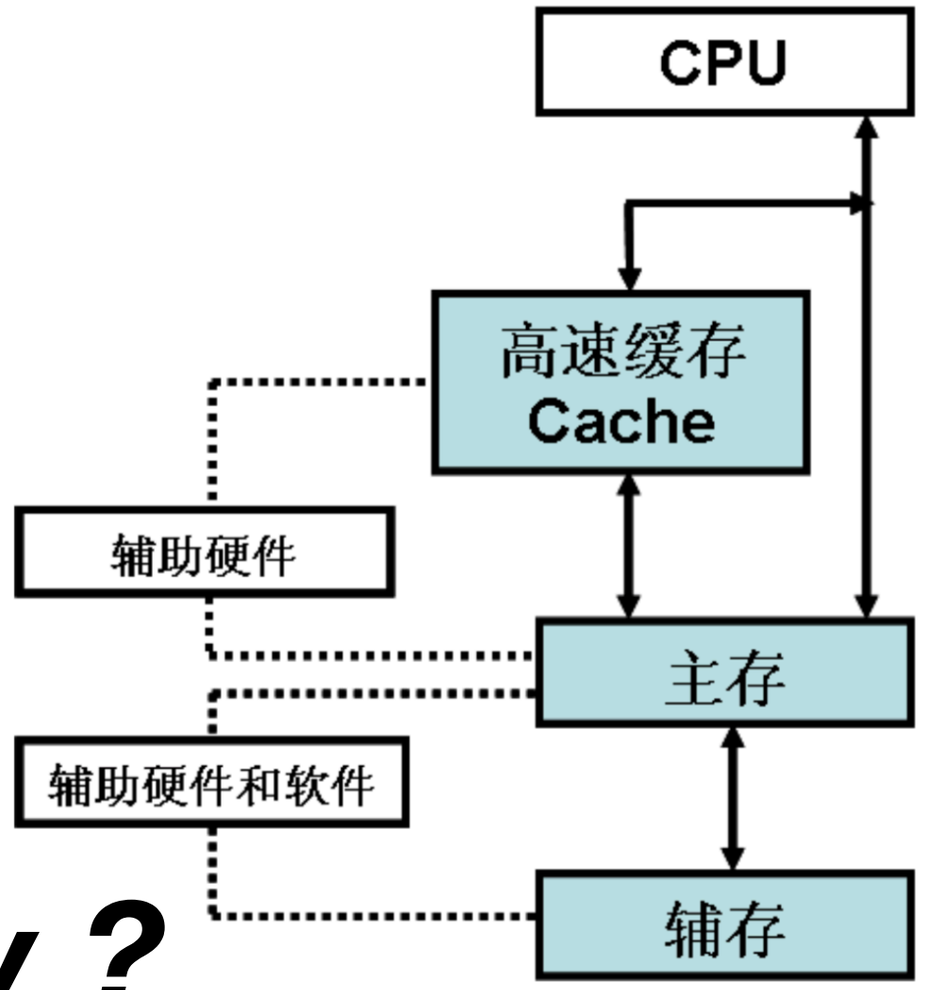
cache-主存-辅存三级存储体系： 三级存储层次

主存-辅存存储层次： 大容量、低成本；

软硬件结合完成

cache-主存存储层次: 高速度、低成本;

纯硬件完成



Then, how ?

解决存储系统三个指标：容量、速度、价格/位的矛盾

7.4 虚拟存储器

- 三种地址映象和变换方法
  - **段式虚拟存储器**
  - **页式虚拟存储器**
  - **段页式虚拟存储器**

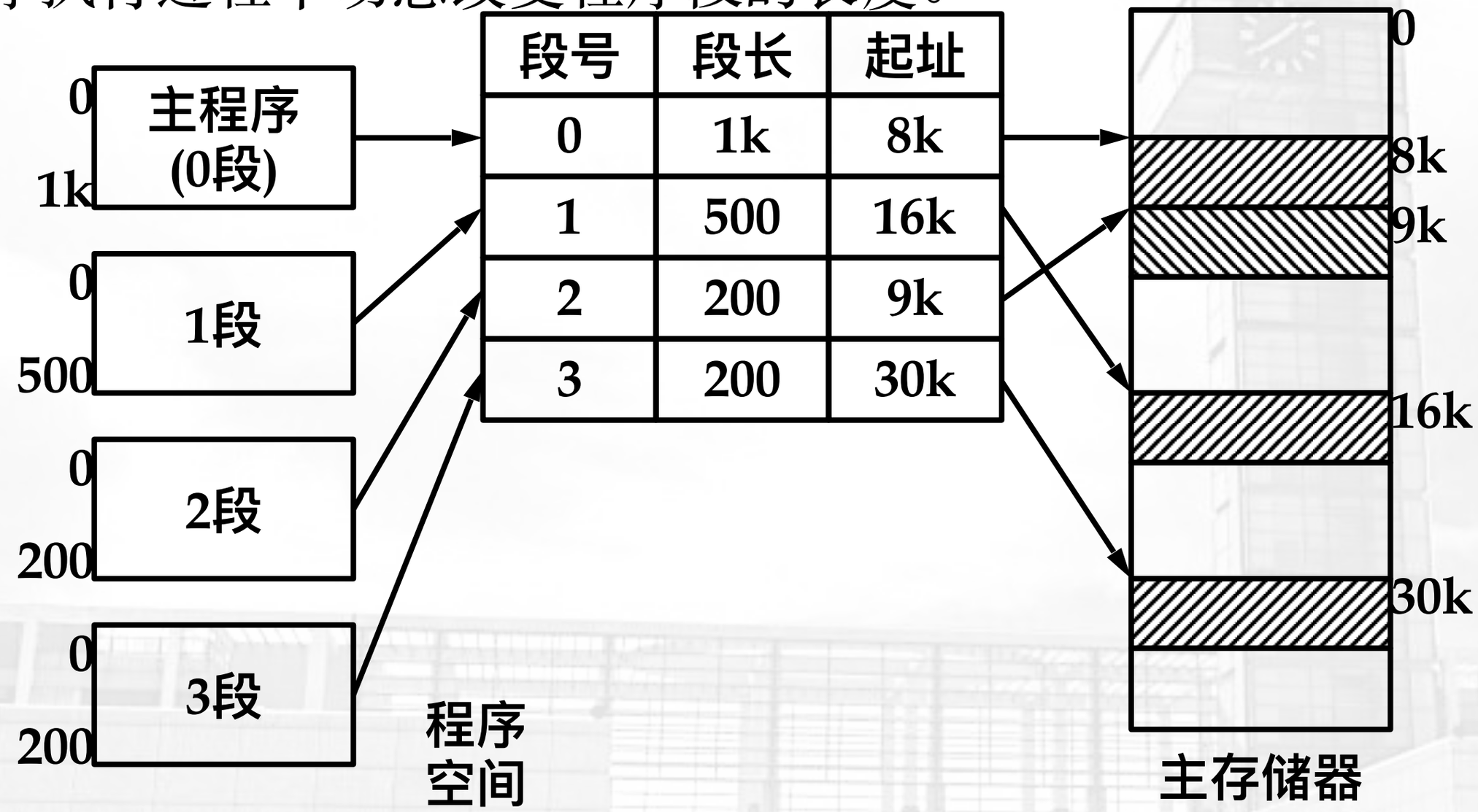
- 段式存储管理方式：

  将程序按逻辑意义分成段，按段进行调入、调出和管理。

段式虚拟存储器

- 地址映象方法：

每个程序段都从0地址开始编址，长度可长可短，可以在程序执行过程中动态改变程序段的长度。



段式虚拟存储器

- 地址变换方法：

① 由用户号找到基址寄存器；

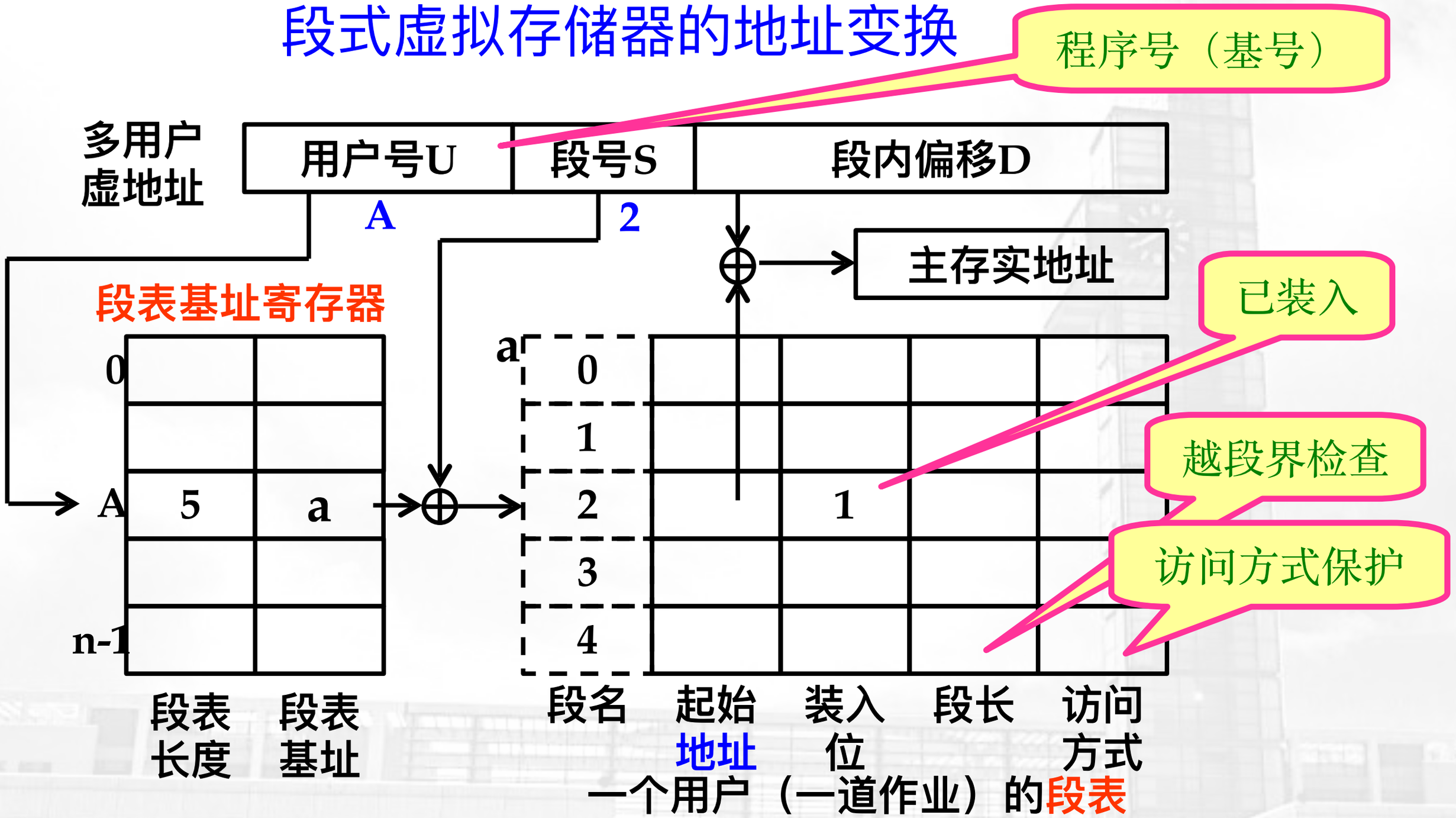
② 从基址寄存器中读出段表起始地址；

③ 把段表起始地址与虚地址中段号S相加得到段表中S段的地址；

④ 把S段的物理内存起始地址与段内偏移D相加就能得到主存实地址。

段式虚拟存储器

段式虚拟存储器的地址变换



段式虚拟存储器

- 主要优点：

① 程序的模块化性能好。

② 便于多道程序共享主存中的某些段。

③ 程序的动态链接和调度比较容易。

④ 便于按逻辑意义实现存储器的访问方式保护。

- 主要缺点：

① 地址变换所花费的时间长，两次加法。

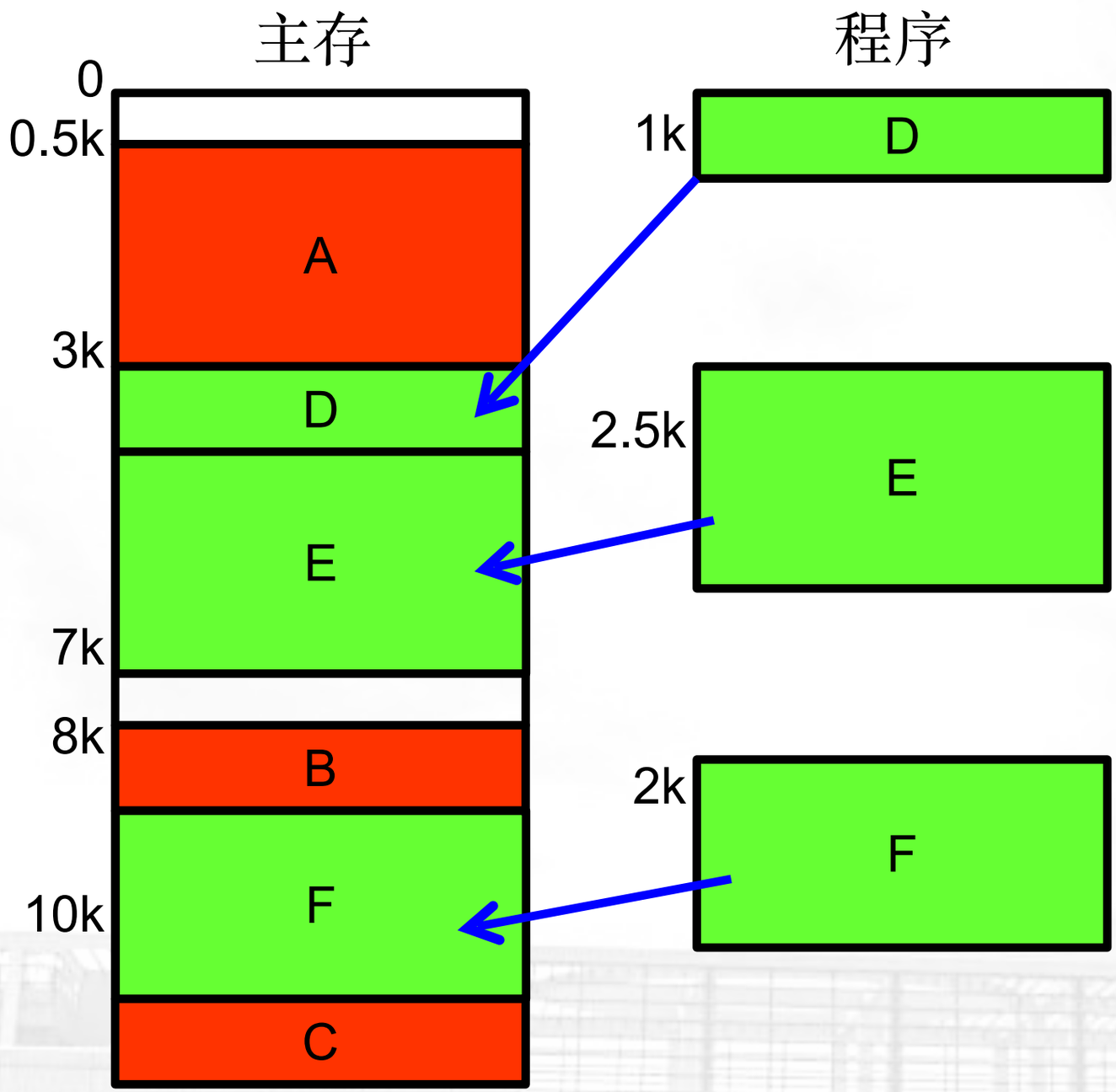
② 段映象表庞大，地址、段长字段太长。

③ 主存储器的利用率往往比较低——存储管理复杂；段间“零头”。

④ 对辅存（磁盘存储器）的管理比较困难。

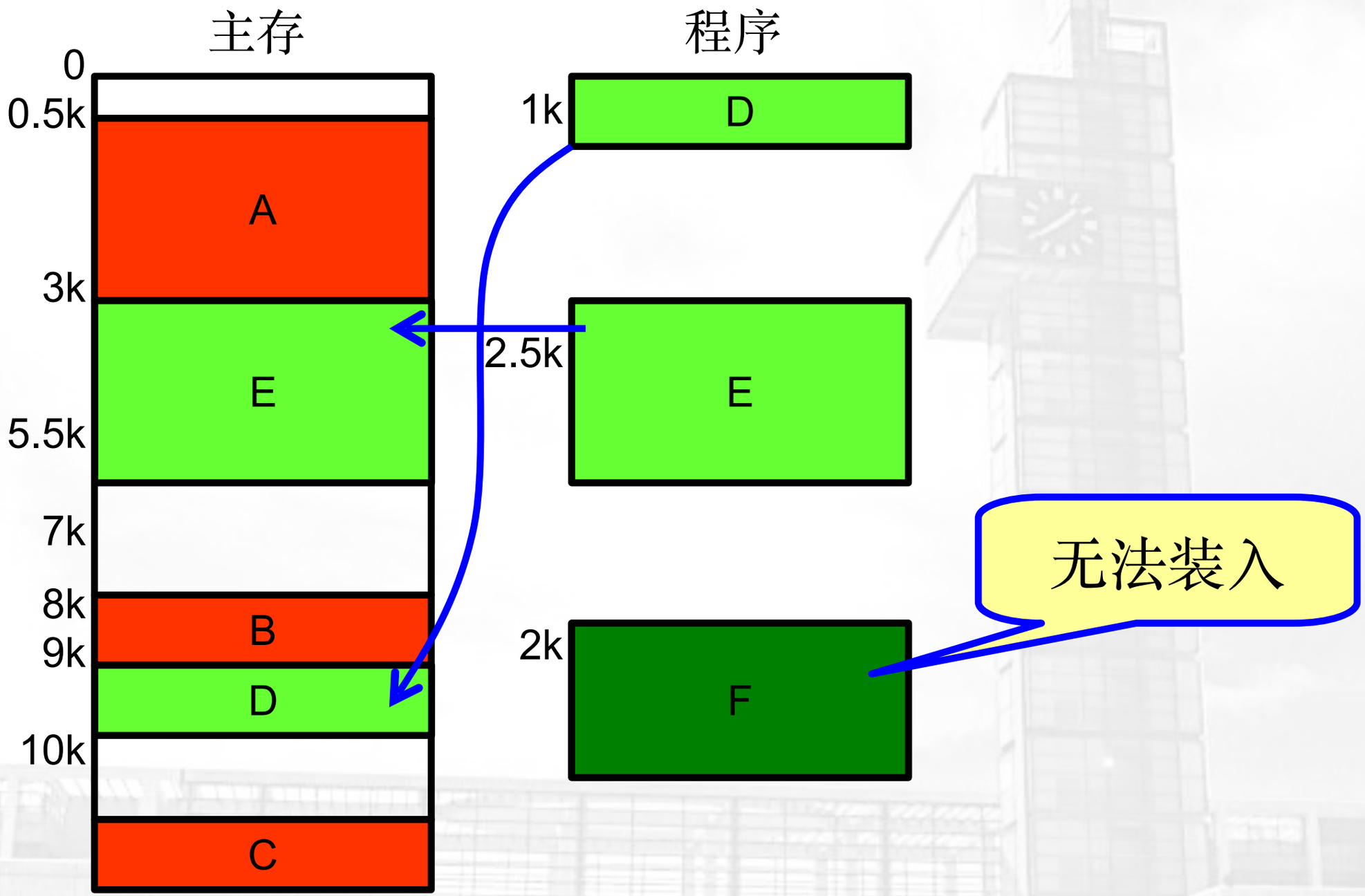
段式虚拟存储器

首先分配法



段式虚拟存储器

最佳分配法



页式虚拟存储器

- 页式存储管理方式：

  将物理空间和VM空间都等分成相同大小的页面(Page)，按页顺序编号，让程序的起点必须处在主存中某一个页面位置的起点上。

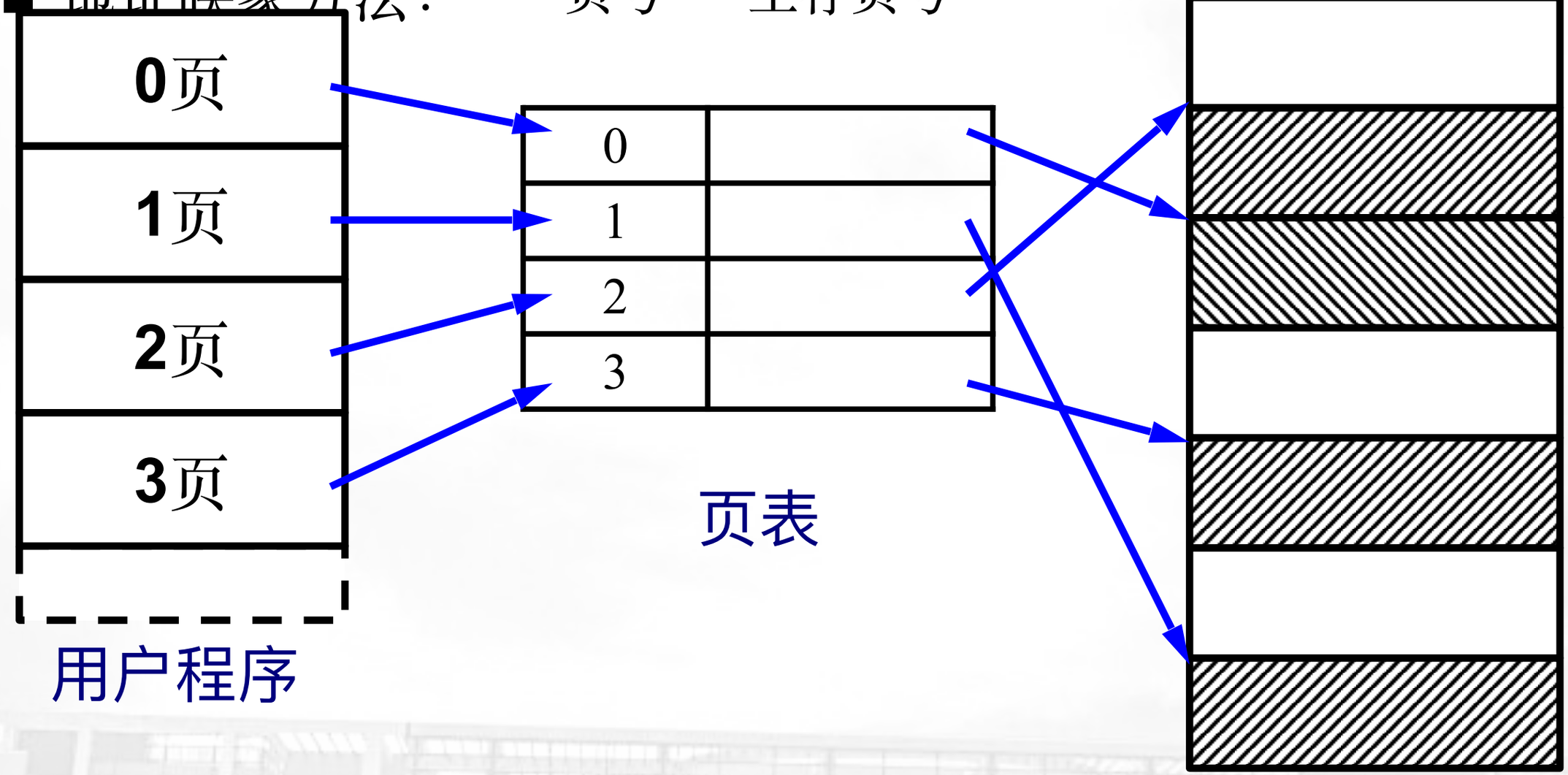
  任一主存单元的地址由页号和页内位移两个字段组成。

页式虚拟存储器

虚页/逻辑页

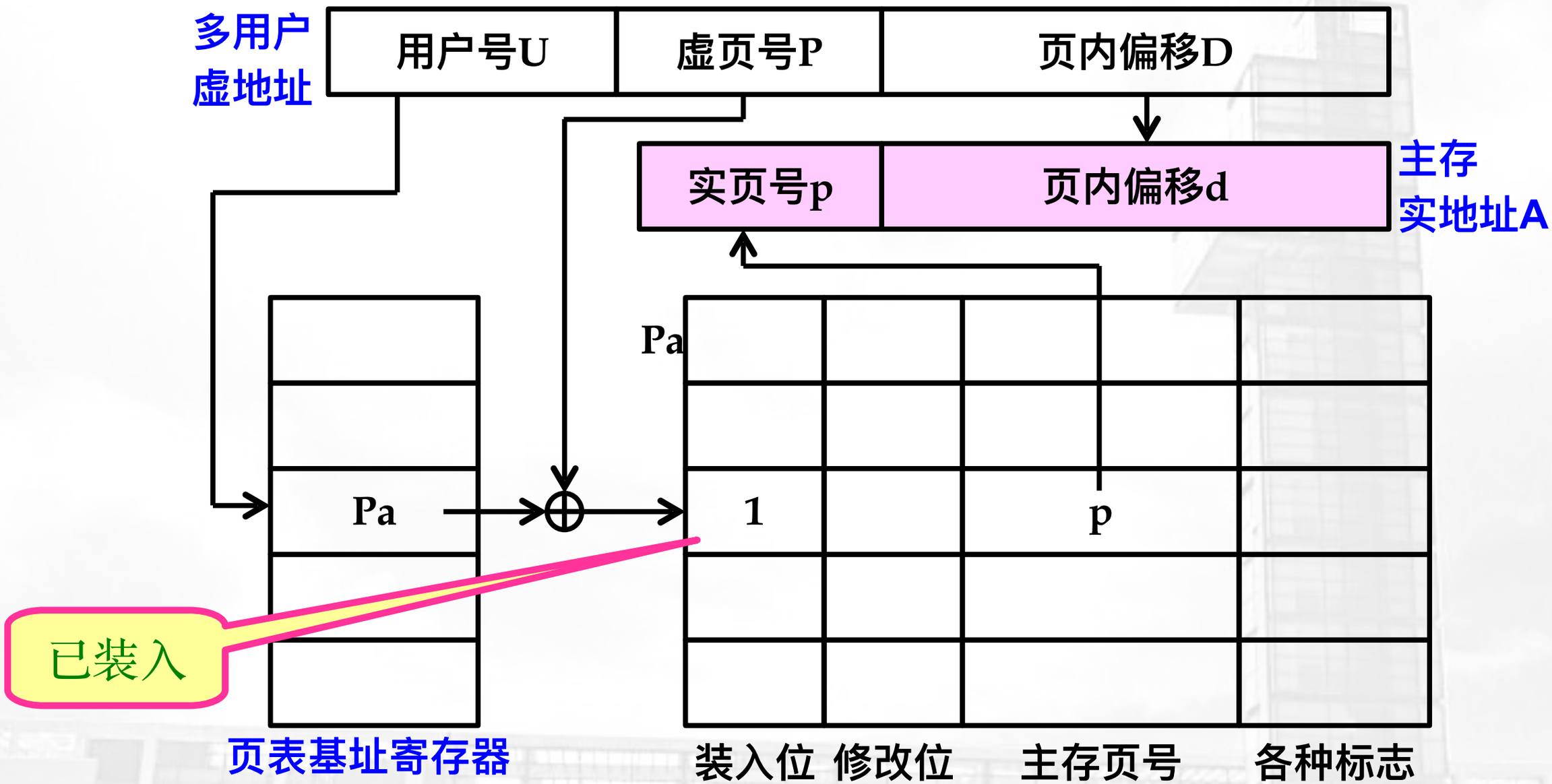
实页/物理页

地址映象方法：　　页号　　主存页号



页式虚拟存储器

- 地址变换方法：



7.4 虚拟存储器　　三、页式虚拟存储器

■主要优点：

① 主存储器的利用率比较高。

② 页表相对比较简单，使用硬件少。

③ 地址变换的速度比较快。

④ 对磁盘的管理比较容易。

■ 主要缺点：

① 程序的模块化性能不好

② 页表很长，需要占用很大的存储空间

例如：虚拟存储空间**4GB**，页大小**1KB**，则页表的容量为**4M**存储字。如果每个页表存储字占用**4**个字节，则页表的存储容量为**16MB**。

段页式虚拟存储器

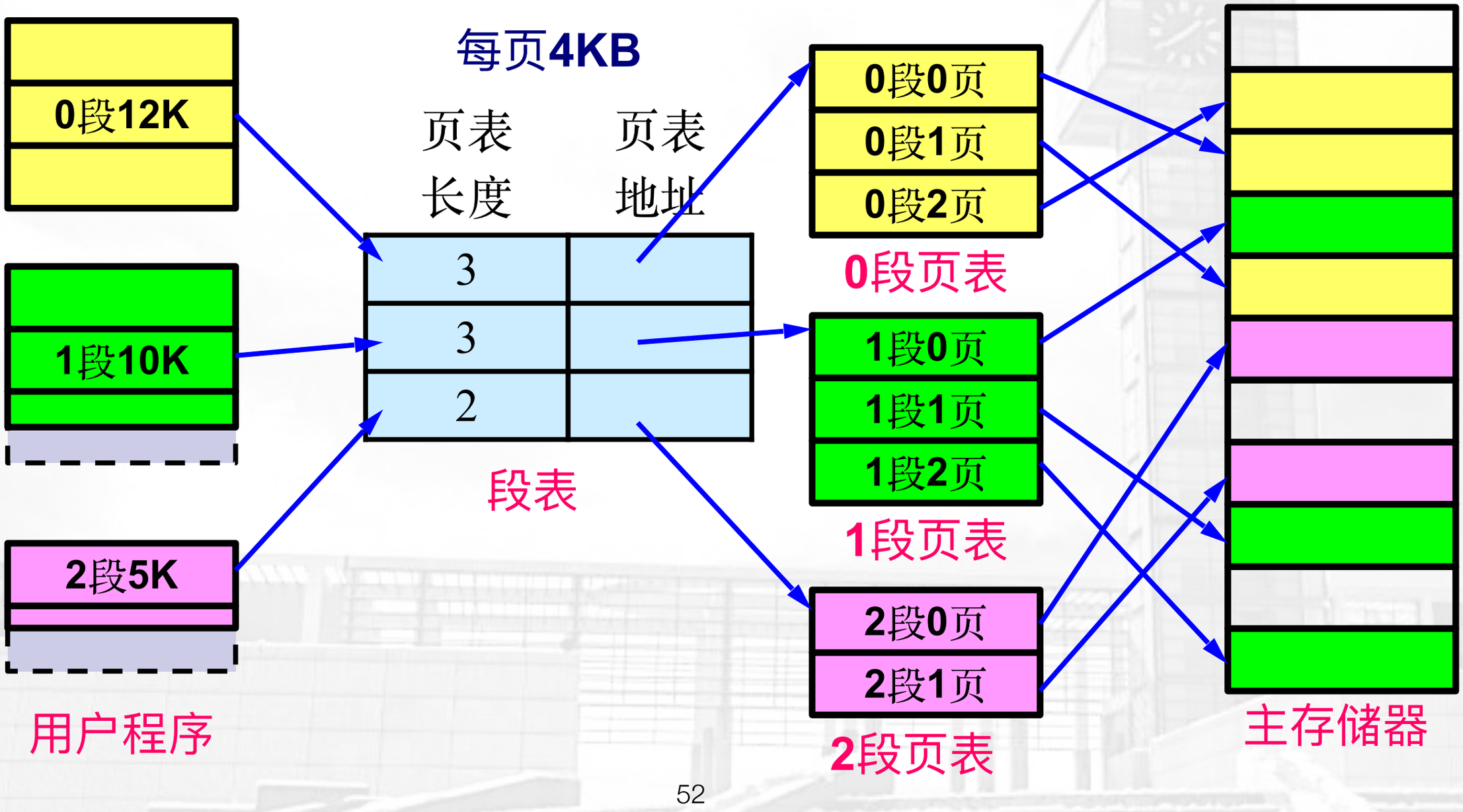
- 段页式存储管理方式：

  将程序按逻辑意义先分成段，再让各段和实主存都机械等分成相同大小的页面。每道程序通过一个段表和相应的一组页表来进行程序在主存空间中的定位。

段页式虚拟存储器

■地址映象方法：

每个程序段在段表中占一行， 在段表中给出页表长度和页表的起始地址，页表中给出每一页在主存储器中的实页号。



■ 地址变换方法：

① 先查段表，得到页表起始地址和页表长度；

② 再查页表找到要访问的主存实页号；

③ 把实页号**p**与页内偏移**d**拼接得到主存实地址。

段页式虚拟存储器

“C” “1” “2”

多用户虚地址A_V

用户号U	段号S	虚页号P	页内偏移D

实页号p	页内偏移d

主存实地址A

段表基址寄存器

	S_A
	S_B
	S_C

程序A段表

S_A

程序C段表

S_C

				a
1	0/1		3	b
				c

装入位 修改位 标志 页表长 页表地址

多用户段表

C_0段

a

C_1段

b

1	p	0/1	

C_2段

c

装入位 实页号 修改位 标志

多用户页表

Linux的内存管理

- 严格来说，是段页式（兼容x86-386构架)
- 但是，所有的页都在段0
  - 因此，Linux实际是页式内存管理

Big Picture of VM

- 映射方式
  - 段、页、段页
- 替换：主存空间不够时置换部分页到外存
- VM实现机制
  - 查表 + Page Fault缺页中断
- 访存过程：映射、查找、读写、替换

虚存地址（来自CPU）

虚页号B | 页内地址W

虚存地址

段/页表

虚存→实存
地址变换

不在内存

内存不够

内存够

在内存

实存
地址

实页号b | 页内地址w

虚存
页置换

替换页

装入页

内存

内存

外存

数据送CPU

VM访存流程

- CPU给出虚拟地址
  - 查段/页表
    - 如果在内存(有效位=0)，访问
    - 如果所需页面不在内存(有效位=0)…
      - CPU发出Page Fault(PF)中断
      - 启动操作系统注册的FP中断处理程序
      - 中断处理程序从外存调数据进内存（文件系统等…)
      - 操作系统更新页表（有效位=1）
      - 操作系统让切换回原进程

VM页替换

- 物理内存不足时，将暂时不用的物理页替换到外存
- 虽然慢，但可以维持系统运行
- 常规策略
  - LRU，最近最少使用
    - 需要为每一页记录时间戳
  - FIFO，先进先出

VM模型下的内存访问

虚存地址(虚存页号 + 页内偏移)

How ?

物理内存地址

访问物理内存地址

VM模型下的内存访问

虚存地址(虚存页号 + 页内偏移)

How ?

1. 找到当前进程的页表/段页

2. 查表 + 计算

物理内存地址

访问物理内存地址

So How to accelerate ?

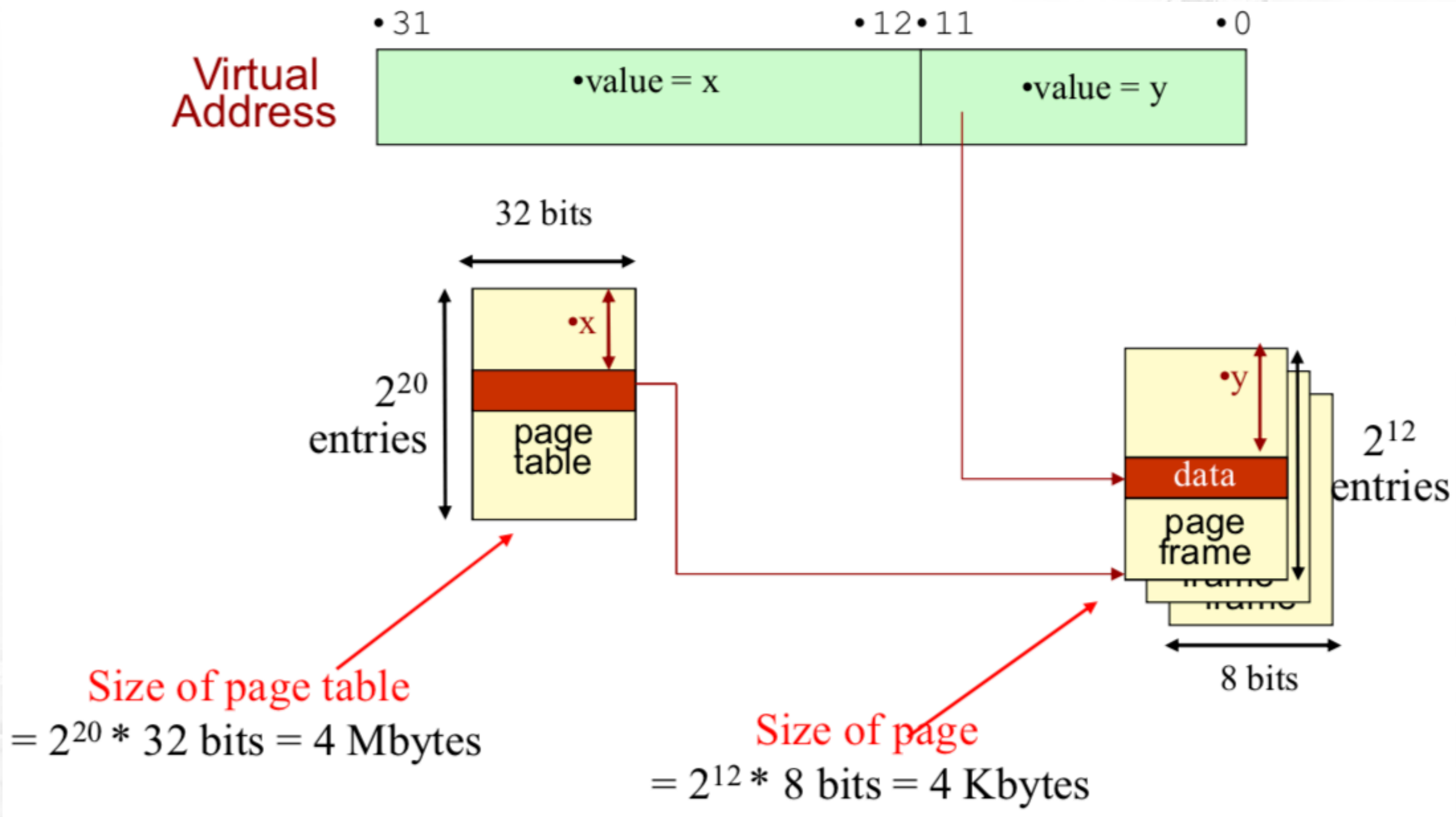
1. 地址转换(or 计算)成为CPU极频繁的工作；
2. 页表被极频繁的访问;
3. so, 大量页表内容进驻cache;
4. cache容量有限，进程指令/数据的cache被挤占；
5. 更可能的情况是，页表体积常远大于cache，使页表本身都无法快速访问；
6. 系统性能严重下降
7. 极端情况：页表都无法完全放在内存中，可能被置换到外存

VM模型下的内存访问

- 优化一：用专用的内存管理单元(MMU)硬件处理VM地址变换
  - MMU是专用硬件
  - 页表被存储在特殊内存位置PTBR

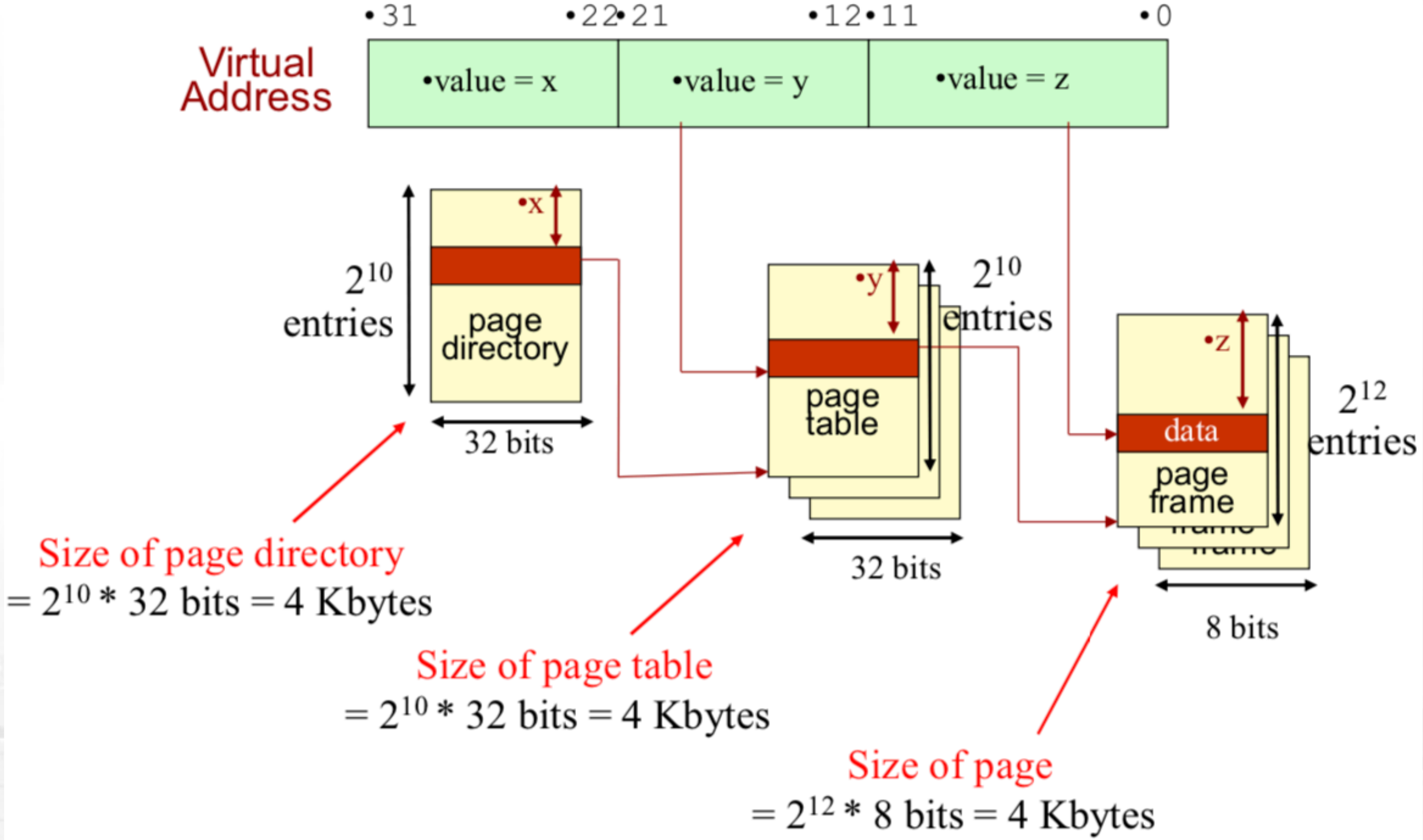
VM模型下的内存访问

- 优化二：单极页表太大，怎么办?



VM模型下的内存访问

- 优化二：两级页表！

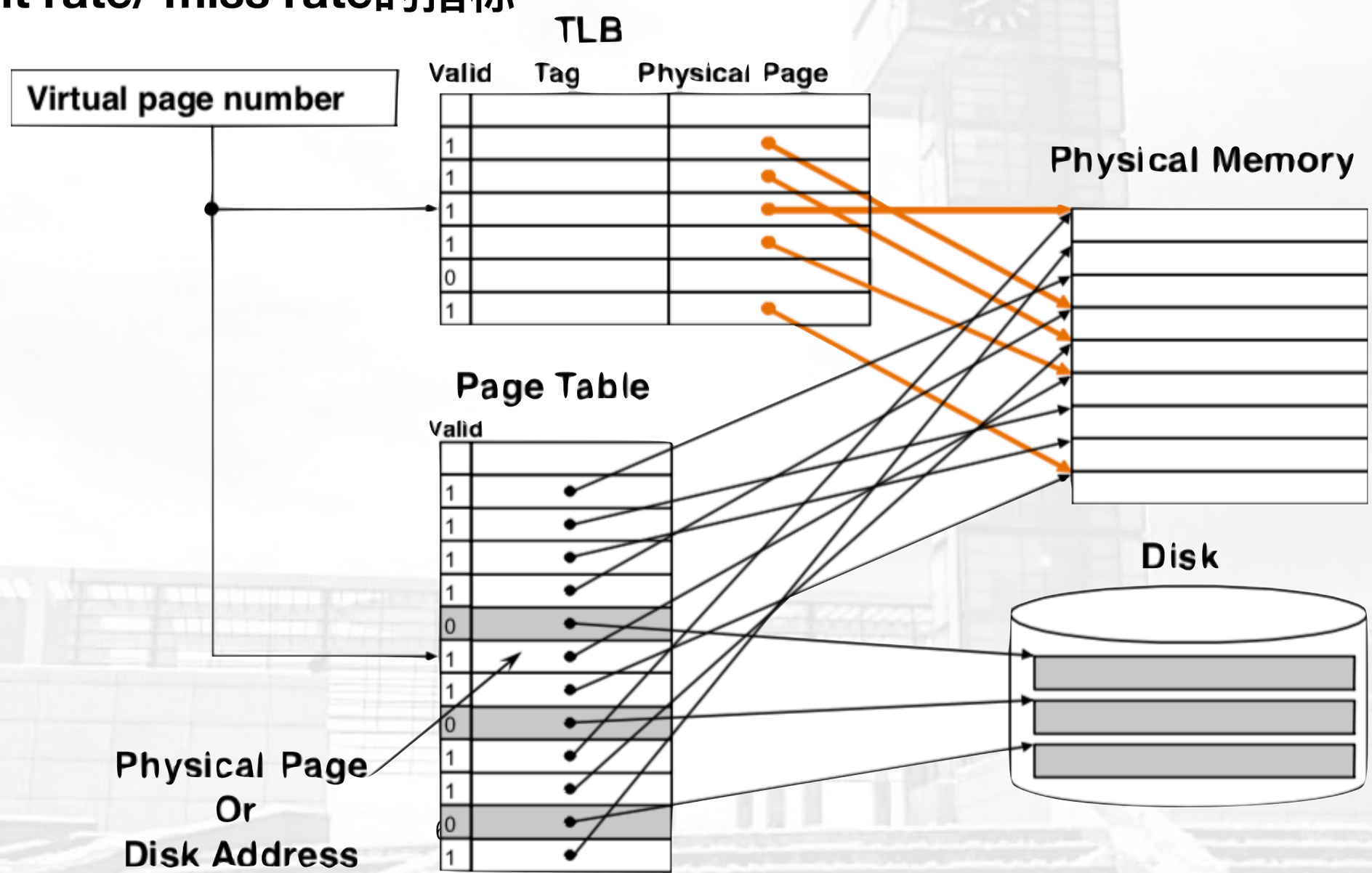


VM模型下的内存访问

- 优化三：页表在内存访问慢，在Cache挤程序空间，怎么办?

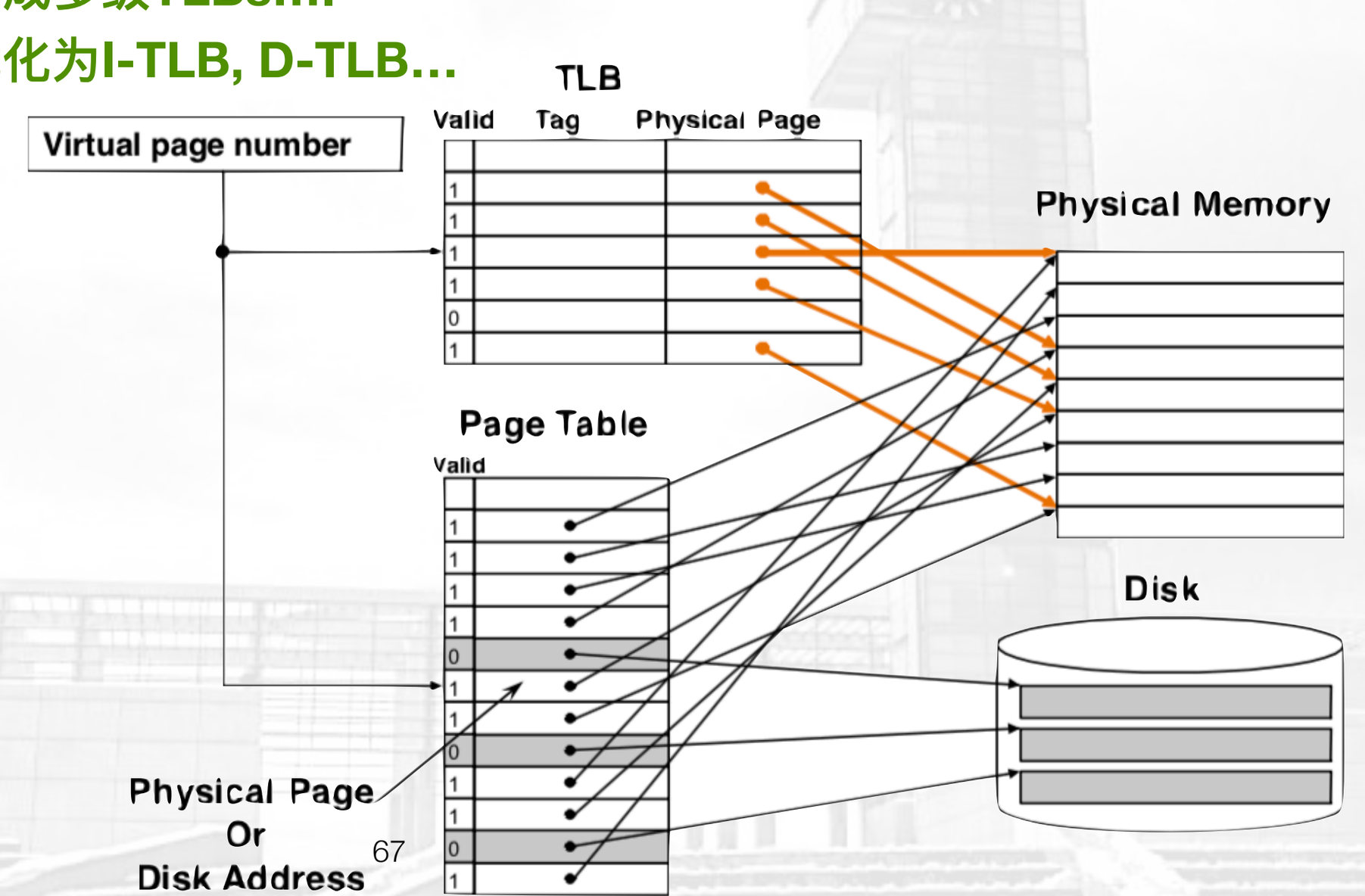
VM模型下的内存访问

- 优化三：给页表加一个专用的Cache——TLB
  - Translation Look-aside Buffers
  - 比L1 Cache的级别更高！
    - 在CPU和Cache之间
  - 同样有hit rate/ miss rate的指标



VM模型下的内存访问

- 优化三：给页表加一个专用的Cache——TLB
  - 因为TLB是个Cache…
    - TLB的映射方式：1/2/4/8路组相联
    - TLB的替换策略：随机、FIFO、LRU
    - TLB也搞成多级TLBs….
    - TLB又优化为I-TLB, D-TLB…



VM模型下的内存访问

- 最终的内存访问流程~

